Does Bangalore Traffic Explain House Prices?

Uncertainty-Aware Price Prediction with Road-Network GNNs and Bayesian Heads

Target Timeline: 2 Months (8-Week Sprint) **Domain:** Urban Computing, Spatial GNNs, Bayesian ML **Target Venue:** Workshop / Short Paper (IEEE/ACM)

1. Executive Summary & Project Vision

Urban real estate valuation models overwhelmingly rely on intrinsic listing attributes—square footage, bedroom counts (BHK), bathroom count, and basic neighborhood name categorical flags. However, housing prices in hyper-growth tech hubs like Bangalore are fundamentally constrained by spatial frictions, commuter bottlenecks, and infrastructural accessibility. This project constructs an end-to-end spatial-economic machine learning pipeline that investigates whether explicit road-network congestion features and spatial graph topologies can significantly improve price predictions and out-of-locality generalization.

> **Core Research Question**
>
> *To what extent do peak-hour commute times to primary tech clusters (Whitefield, ORR/Bellandur, Electronic City) and metro proximity explain residual house price variance beyond tabular features, and can we quantify spatial prediction uncertainty reliably across unseen localities?*

2. Literature Review & Related Work

To position this project for publication, it is essential to ground it within existing literature across three pillars:

- **Hedonic Pricing Models (HPUs):** Classical spatial economics models (Rosen, 1974) evaluate housing as a bundle of structural and locational attributes. Modern extensions incorporate spatial autoregressive models (SAR) or geographically weighted regression (GWR), though they scale poorly with unstructured text or rich locality graphs.
- **Graph Neural Networks in Urban Real Estate:** Recent literature (e.g., Wang et al., 2021; Yao et al., 2018) utilizes spatial networks where nodes represent properties or regions. While graph convolutional networks (GCNs) capture spatial autocorrelation, most existing work ignores dynamic traffic impedances (peak vs. off-peak routing costs).
- **Uncertainty Quantification (UQ) in Spatial Economics:** Point estimates fail in real estate due to market volatility. While Deep Ensembles and Monte Carlo Dropout provide epistemic uncertainty, conformal prediction (MAPIE) offers distribution-free finite-sample guarantees, which is vital when evaluating generalization on unseen localities (leave-locality-out splits).

3. Research Gap & Novelty

Most housing price literature suffers from three major limitations that this project directly addresses:

1. **Static Locality Representations:** Standard datasets treat localities as isolated categorical variables or simple lat-long coordinates, ignoring actual road topologies, bottlenecks, and topological bridges.

2. **Ignorance of Temporal Traffic Dynamics:** Proxies for location are typically Euclidean distances rather than directional, peak-hour congestion matrices toward core employment nodes.
3. **Uncalibrated Point Estimates:** Few models provide trustworthy predictive intervals, especially when extrapolating to newly developed suburban localities.

Proposed Novelty: A dual-component architecture combining a **Road-Network Graph Neural Network (GraphSAGE/GAT)** trained on OpenStreetMap topological embeddings with **Peak-Hour Commute Vectors**, coupled with a **Bayesian/Conformal Head** that outputs calibrated prediction intervals and interpretable price-per-commute-minute elasticities.

4. Comprehensive Data Collection & Engineering Pipeline

Building this dataset requires synthesizing tabular real estate listings with spatial road networks and travel-time matrices. Every data source must be carefully handled.

Data Source	Source Details	License / Terms	Engineering Pipeline & Extraction Method
Kaggle Listing Data	~13,000 Bangalore housing listings (area_type, availability, location, size, total_sqft, bath, balcony, price).	Kaggle Open Datasets Terms (Non-commercial research use).	Clean missing values, parse string formats (e.g., "1 BHK", "2400 sqft"), filter extreme outliers, and map textual locality strings to canonical centroids.
OpenStreetMap (OSM)	Bangalore road network (nodes, edges, speed limits, lengths) and metro station coordinates.	ODbL (Open Database License) — requires attribution.	Extracted via Python library `osmnx` using bounding box or city graph query for drivable networks.
Traffic Travel-Time Matrix	Peak vs. off-peak travel times from locality centroids to major tech hubs (Whitefield, Bellandur, Electronic City, MG Road).	Google Distance Matrix API / TomTom Routing API (Free tier quotas or OSM routing via `osmnx.routing`).	Geocode locality centroids, compute shortest-path travel times or query API during peak hours (09:00 AM) vs. off-peak hours (02:00 PM).

Precision Real Estate to Location Matching Workflow

Because the Kaggle dataset lacks latitude/longitude coordinates and relies on messy string locality names (e.g., "Hebbal", "Hebbal Kempapura", "Hebbal Bangalore"), exact spatial merging requires a robust pipeline:

1. **String Normalization & Fuzzy Matching:** Clean locality strings by removing punctuation, lowercasing, and matching against a canonical master list of Bangalore neighborhoods using `rapidfuzz` (similarity threshold > 85).
2. **Geocoding Centroids:** For each canonical locality, obtain geographic coordinates *(lat, lon)* using Nominatim (OSM) or Google Geocoding API.
3. **Spatial Indexing with Geopandas:** Construct a polygon boundary or Voronoi tessellation around locality centroids to spatially join listings that lack clean text matches to their nearest geographic zone.

4. **Graph Node Alignment:** Snap each locality centroid to the nearest node in the OpenStreetMap road network graph extracted via `osmnx`.

5. System Architecture & Tech Stack

Model Architecture

The system is structured as a hybrid multi-modal architecture:

- **Tabular Feature Encoder:** A feed-forward subnetwork processing structural listing attributes: $x_{tab} = [sqft, bath, balconies, BHK_encoded, property_type]$.
- **Road-Network GNN (Spatial Encoder):** A 2-layer GraphSAGE or Graph Attention Network (GAT) operating on the locality graph $G = (V, E)$ where nodes V are localities and edges E are road segments weighted by peak-hour travel times and distance. This outputs a 32-dimensional locality embedding h_v for each locality.
- **Feature Fusion:** Concatenate tabular embeddings, GNN locality embeddings, and explicit commute-time vectors: $h_{fused} = [h_{tab} \parallel h_v \parallel t_{commute}]$.
- **Uncertainty-Aware Bayesian Head:** A final regression layer with either MC Dropout, a Deep Ensemble (5 models), or a Bayesian Neural Network head predicting $\mu(x)$ and $\sigma^2(x)$ for log-transformed prices.

Tech Stack

- **Core Deep Learning:** PyTorch, PyTorch Geometric (PyG).
- **Bayesian & UQ Modeling:** PyMC / NumPyro (for hierarchical Bayesian regression), MAPIE (for conformal prediction).
- **Geospatial & Graph Processing:** `osmnx`, `geopandas`, `shapely`, `networkx`.
- **Baselines & Utilities:** Scikit-Learn, XGBoost, LightGBM, Pandas, NumPy, Weights & Biases (W&B) for experiment tracking, ArviZ for posterior diagnostics.

6. Complete Algorithmic Blueprint (Which Algorithms to Use & Why)

To successfully execute this project and satisfy publication standards, the pipeline utilizes the following specific algorithms across each phase:

- **Baseline Regression Algorithms:**
  - *XGBoost (Extreme Gradient Boosting):* Used for structured tabular features. It handles non-linear interactions efficiently and serves as our primary non-spatial performance benchmark.
  - *Multilayer Perceptron (MLP):* A standard feed-forward neural network trained via Backpropagation and Adam optimizer to serve as the baseline deep learning model.
- **Spatial Graph Neural Network (GNN) Algorithms:**
  - *GraphSAGE (Sample and Aggregate):* Generates low-dimensional node embeddings by sampling and aggregating neighborhood feature information. Ideal for inductive learning on large road/locality graphs.
  - *Graph Attention Networks (GAT):* Uses masked self-attention layers to assign different importance weights to neighboring localities connected via road networks.
- **Uncertainty Quantification (UQ) Algorithms:**
  - *Monte Carlo (MC) Dropout:* Keeps dropout active during inference to approximate Bayesian neural network uncertainty with minimal computational overhead.

- *Deep Ensembles:* Trains 5 separate models with different random initializations to capture epistemic uncertainty and model disagreement.
- *Conformal Prediction (MAPIE):* Provides distribution-free, finite-sample guaranteed prediction intervals wrapper around point predictors.
- *Hierarchical Bayesian Partial Pooling (PyMC/NumPyro):* Uses Markov Chain Monte Carlo (MCMC / NUTS sampler) to model group-level variations across localities.

7. Experimental Design, Metrics & Ablations

Evaluation Metrics

- **Point Prediction:** Root Mean Squared Error (RMSE), Mean Absolute Percentage Error (MAPE).
- **Uncertainty Calibration:** Negative Log-Likelihood (NLL), 90% Prediction Interval Coverage Probability (PICP), and Continuous Ranked Probability Score (CRPS).
- **Generalization Rigor:** Leave-Locality-Out (LLO) cross-validation to test model transferability to unseen suburban neighborhoods.

Ablation Study Matrix

Experiment ID	Traffic Features Included?	GNN Spatial Embeddings?	Uncertainty Head Type	Primary Evaluation Objective
Exp 1 (Baseline)	No	No (Tabular only)	Deterministic (MSE)	Establish benchmark XGBoost / MLP performance.
Exp 2	Yes (Peak Commute)	No	Deterministic	Measure value-add of explicit commute times.
Exp 3	Yes	Yes (GraphSAGE)	Deterministic	Measure value-add of road-network topology.
Exp 4 (Full Model)	Yes	Yes (GAT)	Bayesian / MC Dropout / Ensemble	Evaluate calibration and out-of-locality error.

8. 2-Month Execution Roadmap (8-Week Sprint)

- **Weeks 1–2 (Data Acquisition & Geo-Pipeline):** Download Kaggle dataset, clean strings, extract OSM road network via `osmnx`, geocode locality centroids, compute distance/travel-time matrices to tech hubs, and build the baseline XGBoost model.
- **Weeks 3–4 (Graph Construction & GNN Baseline):** Construct the locality adjacency graph, implement the 2-layer GraphSAGE/GAT model in PyTorch Geometric, integrate tabular features, and establish initial validation loss curves.
- **Weeks 5–6 (Bayesian Head & Uncertainty Integration):** Implement MC Dropout, Deep Ensembles, and conformal prediction (MAPIE). Run hierarchical Bayesian partial pooling in PyMC/NumPyro to estimate price penalties per commute minute.

- **Weeks 7–8 (Ablations, Evaluation & Paper Writing):** Execute leave-locality-out cross-validation across 3+ random seeds, generate calibration plots and ArviZ posterior diagnostics, and compile the final conference short paper (Introduction, Methodology, Results, Conclusion).